北京理工大学
Beijing Institute of Technology

Spring 2025

操作系统

宋 红

RPC
(Remote Procedure Call)

RPC

- What is RPC?

✓ 远程过程调用

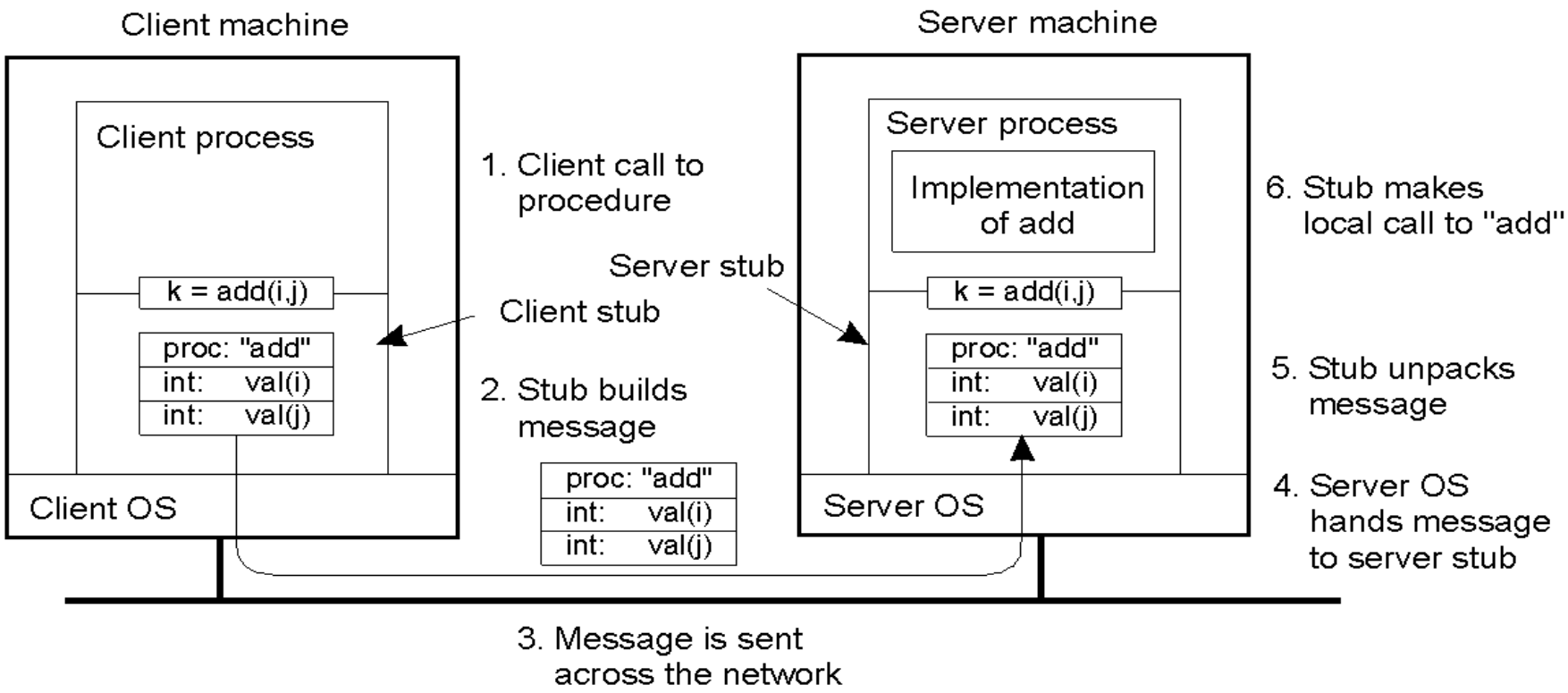
- Why use RPC?

✓ 解决分布式系统中，服务之间的调用问题

✓ 远程调用时，要能够像本地调用一样方便，让调用者感知不到远程调用的逻辑

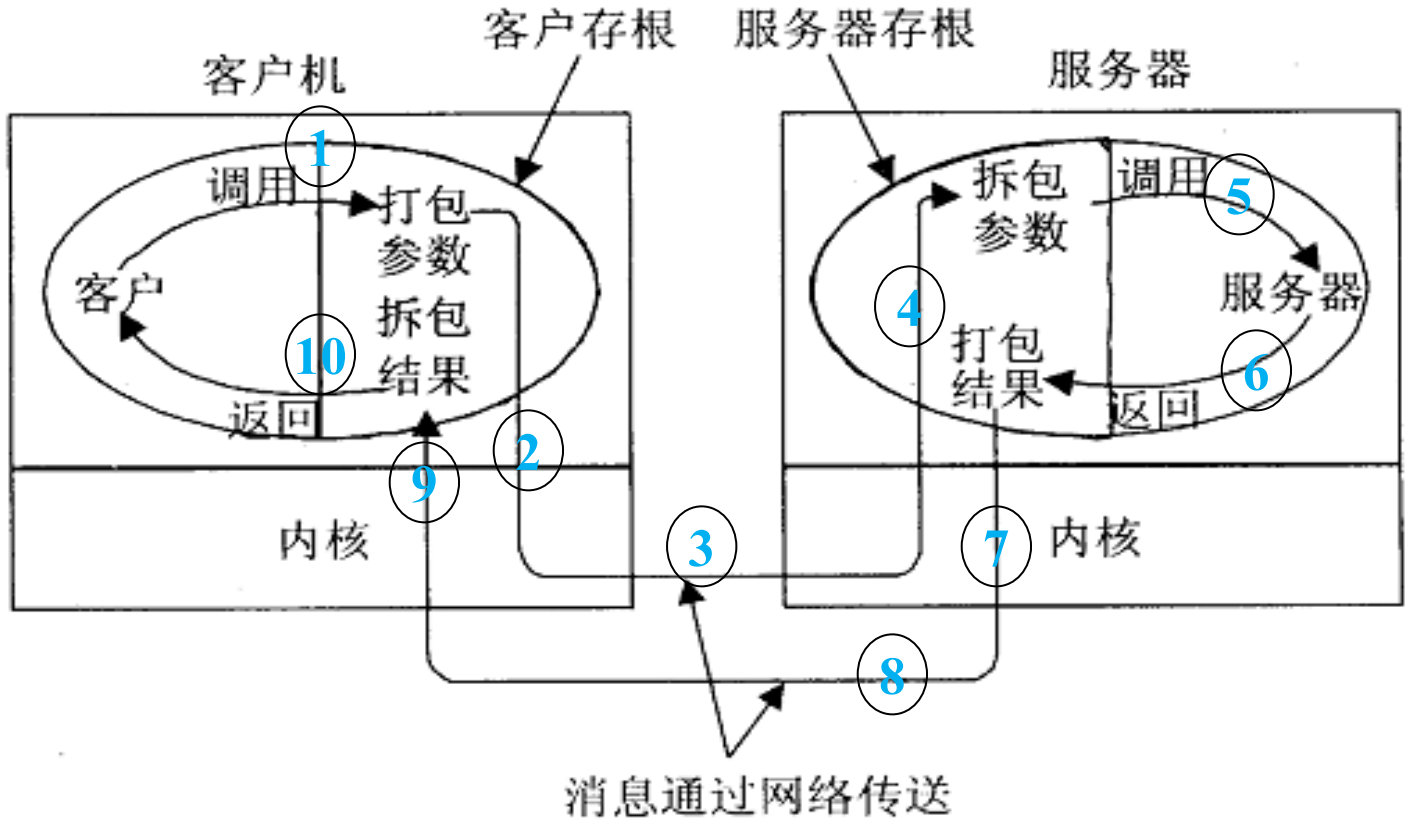
- How to implement RPC?

RPC(1)


Client machine
Server machine
Client process
k = add(i,j)
proc: "add"
int: val(i)
int: val(j)
Client OS
1. Client call to procedure
Server stub
Client stub
2. Stub builds message
proc: "add"
int: val(i)
int: val(j)
Server process
Implementation of add
k = add(i,j)
proc: "add"
int: val(i)
int: val(j)
Server OS
6. Stub makes local call to "add"
5. Stub unpacks message
4. Server OS hands message to server stub
3. Message is sent across the network

RPC(2)

- RPC的步骤


客户存根
服务器存根
客户机
服务器
调用
打包
参数
拆包
结果
客户
返回
拆包
参数
调用
服务器
打包
结果
返回
内核
内核
消息通过网络传送
1
2
3
4
5
6
7
8
9
10

RPC(3)

- Client Stub
  - Piece of code responsible for proxying the remote call as a local call , and packaging the call up as a message.
    - does not ask the operating system to give it data
    - packs the parameters into a message
    - requests that message to be sent to the server
    - following the call to send, the client stub calls receive, blocking itself until the reply comes back

RPC(4)

- Server Stub
  - Piece of code responsible for unpacking the message on the server, and invoking the actual server-side, application-level implementation.
    - the server stub calls receive, blocking itself until the request comes back
    - unpacks the parameters from a message
    - Server is written as a standard procedure
    - Stubs take care of packaging arguments and sending messages

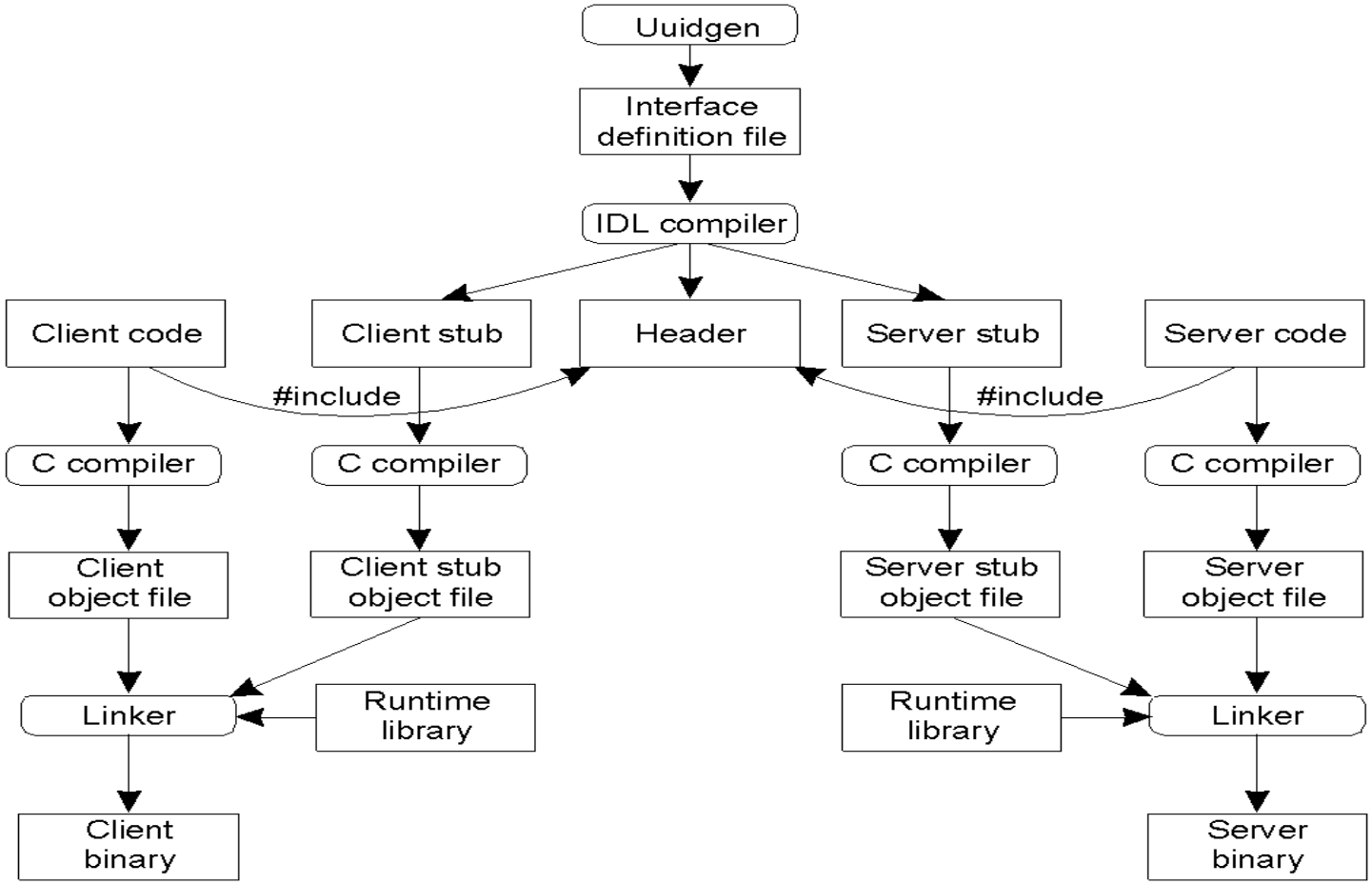
RPC(5)

- Interface Definition Language (IDL)
  - Stub compiler generates stub automatically from specifications in an IDL

```
# include <header.h>

specification of file_server,version 3.1:

    long read(in char name[MAX_PATH],out char buf[BUF_SIZE],
              in long bytes,in long position);

    long write(in char name[MAX_PATH],in char buf[BUF_SIZE],
               in long bytes,in long position);
    int create(in char[MAX_PATH],in int mode);

    int delete(in char[MAX_PATH]);

end;
```

RPC(6)

- Writing a Client and a Server



RPC(7)

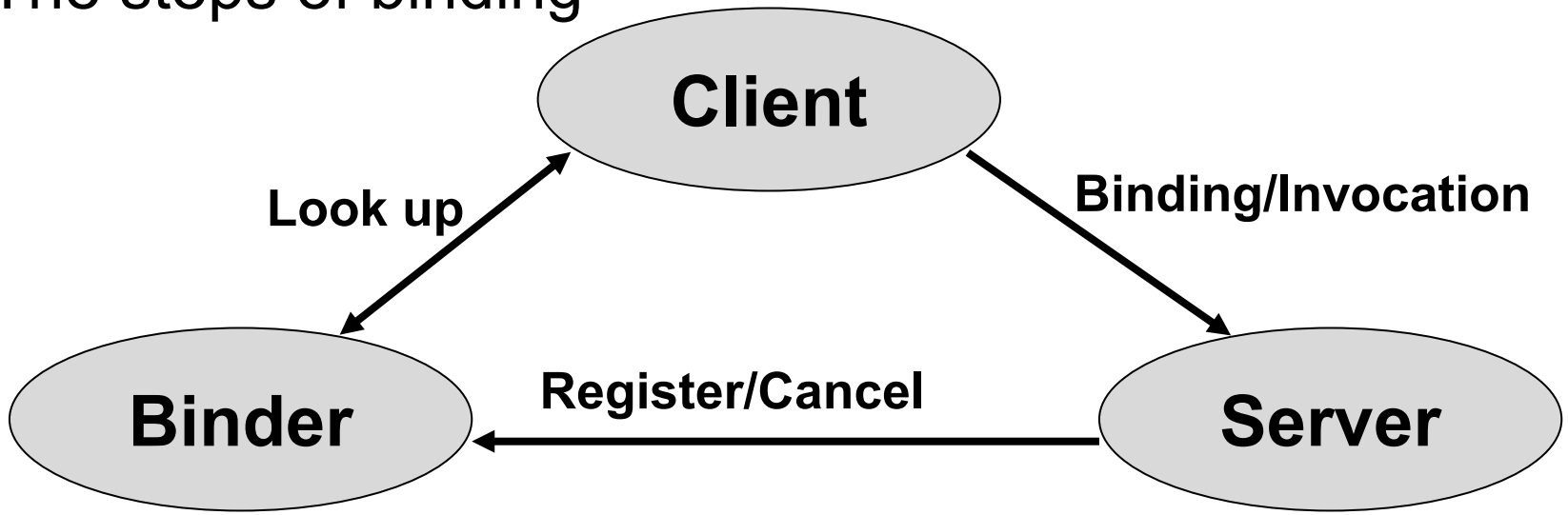
- A UUID (universally unique identifier) ensures that interfaces are immutable.
- IDL is the glue that holds everything together.
- Three files output by the IDL compiler.
  - A header file
    - e.g., interface.h, in C terms.
  - The client stub.
  - The server stub.

RPC(8)

- Binding a Client to a Server
  - Server location is done in two steps.
    - Locate the server's machine.
    - Locate the server (process end point/port) on that machine.
  - How does a client locate a server?
    - Use Bindings
      - Binding: the RPC system can automatically locate the correct server, and subsequently set up the communication between client and server software

RPC(9)

– The steps of binding



- Server
  - Export server interface during initialization
  - Send name, version no, unique identifier, handle (address) to binder

RPC(10)

- Client
  - First RPC: send message to binder to import server interface
- Binder
  - Check to see if server has exported interface
  - Return handle and unique identifier to client

- Example
  - 计算阶乘的RPC实例开发过程

RPC(11)

- 1. 定义接口 (得到RpcFact.idl文件)
  - 创建一个.idl文件, 为应用程序的远程函数定义接口

```
//File: RpcFact.idl
//interface header
[
    uuid (C16F6562-520D-11D0-B338-444553540000),
```

/*universally unique identifier，唯一识别这个接口，它可让服务器用来注册接口，以使客户机可以定位这个特殊接口。uuid由5位数字组成，可以用Microsoft的uuidgen.exe来得到它*/

RPC(12)

```
    version (1.0),  /*指定版本号，那么在同一个网络上就可以同时拥有一个
    RPC接口的不同版本*/
    endpoint("ncalrpc: [myFactorial]")  /*接口的端点，它给出了将会使用的网络
    协议的类型和用来接收接口的请求的地址和端口，ncalrpc 表示Local
    procedure call*/
]
/*interface text, 指定了组成这个接口的函数, 函数类型不支持int 和void* */
interface  rpcfactorial
{
    long  RpcFactorial([in] long nVal);  // [ ]的部分表示属性
}
// end File RpcFact.idl
```

RPC(13)

- 2. 应用程序属性配置文件 (得到RpcFact.acf文件)
  - .acf文件用来指定应用程序的选项，这些属性告诉client stub如何对接口请求的绑定进行处理。

```
// RpcFact.acf
[
   auto_handle  // 相关属性
]
interface  rpcfactorial
{
}
// end File RpcFact.acf
```

RPC(14)

- 3. 用MIDL编译器编译RpcFact.idl文件
  - midl.exe /c_text /ms_text /app_config RpcFact.idl
  - 经过编译之后，会生成下列文件
    - RpcFact.h
    - RpcFact_s.c
    - RpcFact_c.c

RPC(15)

- 4. 开发RPC服务器
  - 定义好了一个接口之后, 需要创建一个响应接口请求的进程, 执行请求的操作并返回所有结果给client. 这些任务由RPC服务器来完成.
  - (1) 执行计算的过程
    - 本例中就是阶乘函数的实现

```
//File: RpcFact.c
#include <windows.h>
#include “RpcFact.h”  // 用MIDL编译器生成
```

RPC(16)

```
long Factorial (long nVal)
{
   long nResult = 1;
   for (; nVal>0; nVal--)
      nResult *= nVal;
   return nResult;
}
// end File RpcFact.c
```

RPC(17)

– (2) RPC服务器实体

- 使RpcFact.c真正对RPC有用的工作是在服务器存根(RpcFacts.c)中实现的，实体内的代码负责建立RPC接口的绑定，并用名称服务程序来注册它们以及侦听RPC请求。

```
// File: RpcFacts.c
#include <windows.h>
#include <iostream.h>
#include <rpc.h>
#include “RpcFact.h”
#include “memstub.h”   //实现RPC函数的存储分配
```

RPC(18)

```
main()
{
    RPC_BINDING_VECTOR*  bindVector;
    RPC_STATUS  lRetVal;
    /*注册所有可行的协议序列, 所支持的协议由在RpcFact.idl的端点定义中的字符串指定*/
    // 参数rpcfactorial_v1_0_s_ifspec定义在RpcFact.h中, 由MIDL编译生成
    if (lRetVal = RpcServerUseAllProtseqsIf(1,
    rpcfactorial_v1_0_s_ifspec, NULL))
    {
        cout<<"Error in
    RpcServerUseAllProtseqsIf"<<lRetVal<<endl;
        return 1;
    }
```

RPC(19)

```
// 注册所支持的接口
if (RpcServerRegisterIf(rpcfactorial_v1_0_s_ifspec, NULL, NULL))
{
cout<<"Error in RpcServerRegister"<<endl;
return 1;
}
// 将接口的绑定输出到名称服务程序
if (RpcServerInqBindings(&bindVector))
{
cout<<"Error in RpcServerInqBindings"<<endl;
    return 1;
}
```

RPC(20)

```
if(RpcServerBindingExport(RPC_C_NS_SYNTAX _DEFAULT, (UCHAR*)"/.:
    /autorpc", pcfactorial_v1_0_s_ifspec, bindVector, NULL))
    {
    cout<<"Error in RpcServerBidingExport"<<endl;
    return 1;
    }
    // 侦听RPC请求
    cout<<"Calling RpcServerListen"<<endl;
    if (RpcServerListen(1, 5, FALSE))
    {
```

RPC(21)

```
        cout<<"Error in RpcServerListen"<<endl;
    return 1;
    }
    return 0;
}// end main
// end File RpcFacts.c

// File: memstub.h
// 以下两个函数在RpcFact.h中已经声明
void __RPC_FAR * __RPC_USER MIDL_user_allocate(size_t len)
```

RPC(22)

```
{    return (new (unsigned char [len]));
}
void __RPC_USER MIDL_user_free( void __RPC_FAR* ptr)
{    delete ptr;  }
// end File memstub.h
```

- (3) 建立服务器
  - 建立服务器时需要确保连接server stub函数(即RpcFact_s.c)编译后的.obj文件， 此外还得添加两个RPC运行库(rpcns4.lib和rpcrt4.lib)。

RPC(23)

- 5. 开发RPC客户端

```
// File: RpcFactc.c

#include <windows.h>

#include <iostream.h>

#include <rpc.h>

#include <stdlib.h>

#include “RpcFact.h”

#include “memstub.h”
```

RPC(24)

```
main()
{
    long nVal;
    cout<<"Calling RPC Factorial"<<endl;
    nVal = RpcFactorial(5);
    cout<<"RpcFact returns: "<<nVal<<endl;
    return 0;
 }
 // end File RpcFactc.c
```